

Day 2 – Android Dynamic Analysis Assignment

ADB and Frida – Standard Practical Tasks

Based on: Dynamic Analysis Android Application Lab Manual

Platform: Windows PowerShell or Ubuntu Terminal

Mode: Individual practical assignment

Target: Instructor-provided vulnerable training APK (e.g., VulDroid / AndroGoat) and authorized Android emulator only

Instructions

1. Complete this assignment using the Dynamic Analysis Android Application Lab Manual as your primary reference. The manual section for each task is given in its Hint.
2. Use only the APK, emulator, and dummy data approved by the instructor.
3. Do not test another person's application, device, account, or data.
4. Frida is used in this assignment only to verify a connection and Java-runtime access. Do not use it to bypass protections or collect secrets.
5. Replace placeholders such as <APK_PATH>, <TARGET_PACKAGE>, <PID>, and <FRIDA_SERVER_BINARY> with the actual values obtained during your own lab session.
6. Record every command you execute, its output, and a short explanation of the output in a single text file. Take a screenshot wherever a task asks for one.
7. If a command fails, record the exact error message and the corrective step you attempted, referencing the manual section used.
8. Convert the completed text file into PDF and upload it to the LMS before the deadline.

Part A: ADB – Application Setup, Information & Logs

Objective: Connect to the emulator, install and identify the target application, inspect its package details, control its execution, check its storage access, and capture/search its runtime logs.

Manual sections to use: Sections 5.1 – 5.8 and 6.1 – 6.6 of the Dynamic Analysis Android Lab Manual.

- **Task 1** — Start your configured emulator and confirm that your workstation can detect and communicate with it over ADB.
Hint: Use the command that lists connected devices. Refer to Section 5.1. Record the device name and status, and take a screenshot.
- **Task 2** — Retrieve four basic device properties: device model, Android version, API level, and CPU architecture.
Hint: Use adb shell getprop with the relevant property names. Refer to Section 5.2 and record the results in a small table.

- **Task 3** — Install the instructor-provided APK on the emulator, then identify its full package name.
Hint: Use `adb install` (quote the path on Windows), then filter the package list with `grep` or `Select-String` using a keyword from the APK name (e.g., `vuldroid` or `goat`). Refer to Section 5.3.
- **Task 4** — Confirm the installed APK's on-device path, then extract key package details such as version, debuggable flag, one permission, and one component (activity/service/receiver).
Hint: Use `adb shell pm path`, then `adb shell dumpsys package` filtered with `grep/Select-String`. Refer to Sections 5.4 and 5.5. Write 'Not observed in the output' for anything missing.
- **Task 5** — Start the application, confirm it is running, note its process ID, and then stop it.
Hint: Use `adb shell monkey` to launch, `adb shell pidof` to check, and `adb shell am force-stop` to close it. Refer to Sections 5.6 and 5.7.
- **Task 6** — With the application running, check whether you can access its private data directory.
Hint: Use `adb shell run-as` with the target package. Refer to Section 5.8. A 'package not debuggable' response is a valid result — record it as such.
- **Task 7** — Capture the application's Logcat output during one normal workflow, then search the saved log for security-related keywords (e.g., password, token, secret, `http://`, `sqlite`).
Hint: Clear old logs first, redirect `adb logcat` output to a file while using the app, then search the file with `grep/Select-String`. Refer to Sections 6.1, 6.2, 6.4, and 6.6. Record up to two observations, or write 'No matching output observed'.

Part B: Frida – Basic Runtime Verification

Objective: Confirm that Frida is correctly set up, start the authorized Frida server, and verify that Frida can attach to the target application's Java runtime.

Manual sections to use: Sections 3.1 or 4.1, and 7.1 – 7.7 of the Dynamic Analysis Android Lab Manual.

- **Task 8** — Confirm your Frida client is installed and identify the emulator's CPU architecture so you can select and start the matching Frida server.
Hint: Check the Frida client version, then read the ABI with `adb shell getprop`, and use the architecture table to pick the matching server family. Refer to Sections 3.1/4.1 and 7.2 – 7.4. Confirm the server process is running before proceeding.
- **Task 9** — Verify that Frida can see the device and attach to the target application, then run the basic Java-runtime test script to confirm Frida attached successfully.
Hint: Use `frida-ps -U` to list processes, then `frida -U -f <TARGET_PACKAGE> -l frida_test.js` with the script from Section 7.7. Record whether 'Frida attached successfully' and 'Java runtime is available' were shown.

Deliverables

- All commands executed and their outputs, recorded in the order performed, in a single text file.
- All screenshots requested in Part A and Part B, numbered and referenced in the text.
- Completed tables for emulator information, package information, and Logcat/security-keyword observations.
- Frida server/client version match and connection verification results.
- A short conclusion of 120–150 words summarizing what was learned and any issues encountered.
- Convert the completed text file into PDF and upload it to the LMS before the deadline.